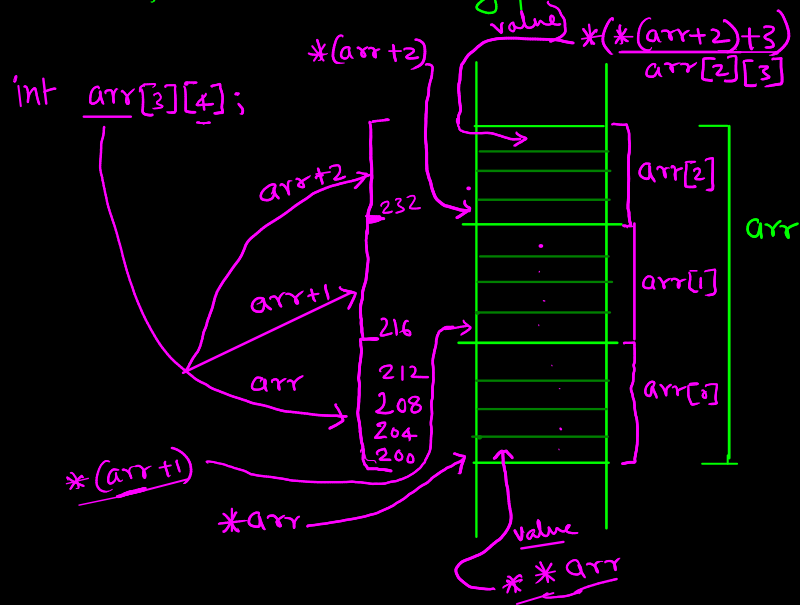


★ Pointer to an array (Array pointer):-

- ✓ - It points to entire array.
- ✓ - It's scale factor depends on type of elements in the array and no. of elements in array being pointed.



★ Referencing and dereferencing array name:-

// Assume size of int to be 4.

`int arr1[4];` // starting address 100
`int arr2[5][6];` // starting address 1000

	at 100	
<code>arr1</code>	pointer to int	4
<code>*arr1</code>	int value at 100	
<code>&arr1</code>	pointer to array of 4 int at 100	16
<code>arr2</code>	pointer to array of 6 int at 1000	24
<code>*arr2</code>	pointer to int at 1000	4
<code>**arr2</code>	int value at 1000	
<code>&arr2</code>	pointer of int array of [5][6] at 1000	120

↑
 *
 ↓

Avoid
`(&arr2+1)`
 points to non-existent byte of [5][6].

int arr3[7][8][9]; // starting address 1000

arr3	pointer to int array of [8][9] at 1000	$8 \times 9 \times 4$
*arr3	pointer to int array of [9] at 1000	9×4
**arr3	pointer to int at 1000	4
***arr3	int value at 1000	—
&arr3	Pointer to int array of [7][8][9] at 1000	$7 \times 8 \times 9 \times 4$

$$\underline{\text{arr3} + 2} \quad 1000 + 2 \times (8 \times 9 \times 4)$$

$$(\underline{* \text{arr3} + 2}) \quad 1000 + 2 \times (9 \times 4)$$

$$(\underline{*(\text{arr3} + 2)} + 3) \quad [1000 + 2 \times (8 \times 9 \times 4)] + 3 \times (9 \times 4)$$

Array name is like a constant pointer to first element of the array

↑ ① No memory to store pointer
Imp ② Referencing & deferencing is different.

int arr3[2][3][4];

int *ptr1, (*ptr2)[4], (*ptr3)[3][4], (*ptr4)[2][3][4] ;

↳ pointer to
int array of
[4]

↳ pointer to
int array
of [3][4]

↳ pointer to int
array of
[2][3][4]

X ptr4 = arr3;
✓ ptr4 = &arr3;
✓ ptr3 = arr3;
~ ptr2 = *arr3;
✓ ptr1 = **arr3;

X ptr4 = &ptr3; → pointer to pointer to int
array of [3][4]
X ptr3 = &ptr2;
X ptr2 = &ptr1;

✓ ptr2 = *ptr3;

int i, *ptr = &i;
int **ptr2 = &ptr3;

